1 | Introduction

The Immersed Boundary (IB) method is a general numerical framework for studying fluid-structure interaction. It was originally developed by Charles Peskin to simulate blood flow in the heart, but has since been applied to a wide variety of problems in biofluid dynamics and beyond.

The fundamental approach of the IB method is to approximate the fluid equations on a fixed Eulerian grid (usually Cartesian), while tracking the flexible immersed boundary using a set of Lagrangian points that move with the fluid. The interaction between the fluid and the structure is mediated by the Dirac delta function, which allows forces to be spread from the Lagrangian boundary to the Eulerian fluid grid, and velocities to be interpolated from the fluid grid to the boundary points. This avoids the need for complex body-fitted mesh generation, making the method particularly suitable for problems with large structural deformations or complex geometries.

1.1 Mathematical Formulation

The equations of motion for the fluid are the incompressible Navier-Stokes equations:

$$\rho\left(\frac{\partial \mathbf{u}}{\partial t} + \mathbf{u}\cdot\nabla\mathbf{u}\right) = -\nabla p + \mu\nabla^2\mathbf{u} + \mathbf{f} \tag{1.1}$$

$$\nabla\cdot\mathbf{u} = 0 \tag{1.2}$$

where $\mathbf{u}(\mathbf{x},t)$ is the fluid velocity, $p(\mathbf{x},t)$ is the pressure, ρ is the fluid density, and μ is the dynamic viscosity. The term $\mathbf{f}(\mathbf{x},t)$ represents the force density applied by the immersed boundary on the fluid.

The interaction between the fluid and the immersed boundary is described by the following integral equations:

$$\mathbf{f}(\mathbf{x},t) = \int_\Gamma \mathbf{F}(s,t)\delta(\mathbf{x}-\mathbf{X}(s,t))ds \tag{1.3}$$

$$\frac{\partial \mathbf{X}}{\partial t}(s,t) = \mathbf{u}(\mathbf{X}(s,t),t) = \int_\Omega \mathbf{u}(\mathbf{x},t)\delta(\mathbf{x}-\mathbf{X}(s,t))d\mathbf{x} \tag{1.4}$$

Here, $\mathbf{X}(s,t)$ denotes the position of the immersed boundary parameterized by the curvilinear coordinate s, and $\mathbf{F}(s,t)$ is the force density on the boundary. The Dirac delta function $\delta(\mathbf{x})$ mediates the interaction between the Eulerian fluid variables and the Lagrangian boundary variables.

1.2 Numerical Methods

The numerical implementation typically involves answering the following on each time step:

1. **Force computation**: Calculate the boundary force $\mathbf{F}(s,t)$ from the boundary configuration $\mathbf{X}(s,t)$ using the constitutive laws (e.g., Hooke's law).
2. **Force spreading**: Spread the boundary force to the fluid grid using the discretized delta function to obtain $\mathbf{f}(\mathbf{x},t)$.
3. **Fluid solver**: Solve the Navier-Stokes equations on the Eulerian grid to update the velocity field $\mathbf{u}$ and pressure p. This is often done using a projection method or a fractional step method.

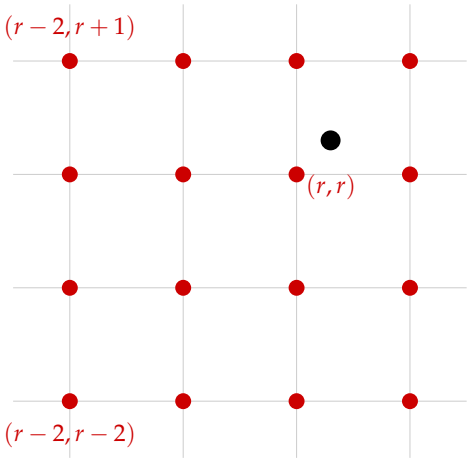
4. **Velocity interpolation**: Interpolate the new fluid velocity to the boundary points applied to update their positions $\mathbf{X}(s,t)$.

1.3 The Interpolation Function

Given the material position X we want to obtian the velocity of fluid at that point. [1] This process is directly interpreted by

[1] *Note that X usually doesn't lies on the grid point.*

$$\mathbf{u}(\mathbf{X}(r,s,t),t) = \int \mathbf{u}(\mathbf{x},t)\delta(\mathbf{x}-\mathbf{X}(r,s,t))d\mathbf{x} \tag{1.5}$$



However in the discrete case we would need the discrete Delta Function.

In the figure on the left, black point is the material point and red points are the grid points. The δ function average all the velocity at the red points.

Something to pay attention is that in MATLAB, the convention for a matrix to represent a grid of point is to have lay points with same x coordinat in the **same row** This may be different from our intuition. However, This is reasonable since we normally define the matrix for storing points to have size

$$N_x \times N_y$$

So it has N_x rows.

1.4 Discrete Delta Function

A commonly used discreted delta function uses a 4×4 grid. Thus the 1D delta function $\phi(x)$ should support on four points. We have the following postulates:

1. $\phi(r)$ is continuous for all real r
2. $\phi(r) = 0$ for $|r| \geq 2$
3. The following gurantee there is no difference between odd index grid point and even index grid point. Since they never exchange information between each others

$$\sum_{j \text{ even}} \phi(r-j) = \sum_{j \text{ odd}} \phi(r-j) = \frac{1}{2} \text{ for all real } r$$

4. The delta function should centered at the origin

$$\sum_j (r-j)\phi(r-j) = 0 \text{ for all real } r$$

5. The L_2 norm shoudl be given.

$$\sum_j (\phi(r-j))^2 = C \text{ for all real } r$$

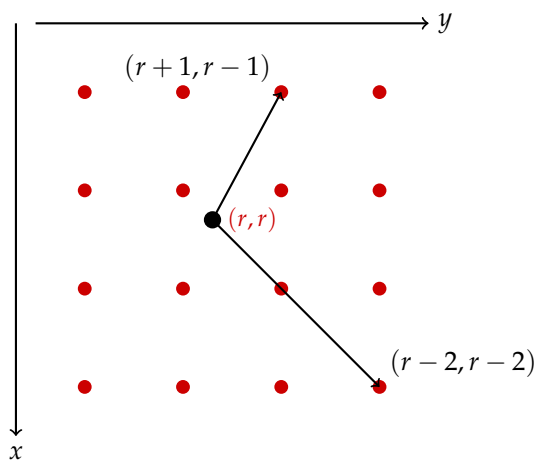
Here $r \in [0,1)$. This is a standard **4 point delta function**. We can also have 6 points delta function for higher accuracy. C can be determined by setting $r = 0$. We get

$$\phi(r) = \frac{3-2r+\sqrt{\Delta}}{8} \qquad \phi(r+1) = \frac{3-2r-\sqrt{\Delta}}{8}$$

and

$$\phi(r-1) = \frac{1+2r+\sqrt{\Delta}}{8} \qquad \phi(r-2) = \frac{1+2r-\sqrt{\Delta}}{8}$$

Here $\Delta = 1+4r-4r^2$.



However, in MATLAB, grid points are stored in a particular way. Entries with same x coordinate is in same row and $(0,0)$ is at the upper-right corner.

Since the argument for ϕ is the position vector of grid point. The value for point $(r-2, r-2)$ is actually at the lower right as shown in the picture on the left. [2]

[2] *I made a mistake here when writing the function* `twoD_delta_function`. *The order is incorrect. Time: 251212*

1.5 Pressure in Navier Stokes

In the Navier Stokes equation we have

$$\rho \frac{\mathrm{D}\mathbf{u}}{\mathrm{D}t} + \nabla p = \mu \nabla^2 \mathbf{u} + \mathbf{f} \quad (1.6)$$

where $\mathbf{f}$ is the force density.

$$\hat{p} = -\frac{i\rho, dx}{\Delta t} \cdot \frac{s_1 \hat{w}_1 + s_2 \hat{w}_2}{s_1^2 + s_2^2} \quad (1.7)$$